

COMPUTER NETWORKS

COURSE PROJECT

Private Network Service Platform

A Two-Phase, Team-Based Local Networking Project

Team Size	Target Environment	Deployment
Maximum 4 Students	Up to 4 macOS Laptops	Fully Local No Cloud Required

PHASE 1 Build & Observe	PHASE 2 Harden & Recover	FINAL Explain & Defend
DNS · HTTP(S) · TCP · TLS · Load Balancing · Packet Analysis	Resilience · Service Isolation · DNS Failover · Controlled	Live Demo · Packet Evidence · Individual Viva

Core Principle:

The application stays simple the network is the project.

1. Project Purpose & Overview

This project gives you hands-on experience with how a real network request travels from a client to a server and back. You will design and build a small private service environment from scratch no cloud, no pre-configured servers using your own laptops connected on a local network.

By the end of this project, a client machine on your team's network will:

- Type a private domain name (e.g., app.team1.test) into a browser or curl command
- Resolve that name through your team's own DNS server
- Establish a secure HTTPS connection through your team's reverse proxy
- Receive a response from one of two backend application servers
- Allow you to observe every step of this journey using packet capture tools

2. What You Will Learn

This project directly maps to course topics you have studied in class. After completing both phases, every team member should be able to:

- **Translate classroom theory into a working network configuration**
 - *You will not just read about DNS or TCP you will configure and run them.*
- **Explain where each protocol fits in a real request**
 - *You must show where DNS, TCP, TLS, HTTP, caching, and load balancing each play a role in a single client request.*
- **Use network tools to prove what is happening**
 - *curl, dig, Wireshark, and browser developer tools are your evidence. Saying "it works" is not enough you must show why it works.*
- **Diagnose failures by isolating layers**
 - *When something breaks, you must identify whether the problem is at the DNS layer, the TCP layer, the TLS layer, or the application layer.*
- **Work as a team while understanding the whole system**
 - *Every team member should be able to explain any part of the project, not just the part they personally configured.*

3. Project Constraints & Ground Rules

Read these carefully. They exist to keep the project achievable and fair across all teams.

Constraint	The Rule
Team Size	1 to 4 students. Maximum 4. Teams of 2–3 may combine machine roles.
Hardware	Up to 4 macOS laptops. No dedicated server or cloud VM is required.
Network	All machines must connect to the same private Wi-Fi/LAN a lab Wi-Fi, home router, or controlled hotspot works fine.
Cloud Hosting	NOT required. Everything must be demonstrable locally on your machines. Cloud concepts may be discussed but not used for the running system.
Admin Access	At least the DNS machine and the edge machine need administrator access to install software and change network/DNS settings.
Application Complexity	Keep backend applications simple. A plain REST API returning JSON is perfectly acceptable. The network, not the app, is what is being evaluated.
Port Restrictions	If your machine does not allow binding to ports 80/443, use 8080/8443 instead. No marks will be deducted for this substitution.
Private Domain	Use the reserved .test namespace (e.g., team1.test). Do NOT use .local it conflicts with macOS mDNS and will cause problems.
Suggested Tools	Homebrew, dnsmasq, nginx, OpenSSL, Python or Node.js, curl, dig / nslookup, Wireshark.

4. Recommended Team Architecture

Each machine in your team takes on a specific network role. This is not a software architecture – it is a network topology. The diagram below shows how the four roles connect to each other.

Machine	Primary Role	Services Running	What It Represents
Mac 1	Private DNS Server + Test Client	dnsmasq, dig / nslookup, curl / browser	Managed DNS service (like Route 53)
Mac 2	Edge / Reverse Proxy + Load Balancer	nginx, TLS certificate, load balancer config	Cloud load balancer / CDN edge node
Mac 3	Backend Server A	Simple HTTP/REST application (port 3001)	Application server instance A
Mac 4	Backend Server B + Test Client	Simple HTTP/REST application (port 3002), curl / browser	Application server instance B

The request flow through your architecture looks like this:

Client (Mac 1 or Mac 4) → DNS Query (Mac 1) → HTTPS Request (Mac 2 / nginx) → Backend A (Mac 3) or Backend B (Mac 4)

5. Two-Phase Project Structure

The project is divided into two phases. Phase 1 must be completed and demonstrated before Phase 2 begins. Phase 2 extends the same infrastructure – you do not rebuild from scratch.

	Phase 1 – Build & Observe	Phase 2 – Harden & Recover
--	---------------------------	----------------------------

Focus	Build the core network infrastructure and prove it works using tools.	Add resilience, service isolation, DNS failover, and controlled failure diagnosis.
End Result	A complete local service reachable through a private domain name with full packet evidence of the DNS → TCP → TLS → HTTP flow.	A more robust network that continues working under selected failures and can be systematically diagnosed when faults are injected.
Gate	Phase 1 is complete when a client resolves app.teamX.test, connects over HTTPS, and receives responses from both backends through the load balancer.	Phase 2 is complete when backup DNS, TTL behavior, service isolation, HA failover, and at least one troubleshooting challenge are demonstrated.

6. Phase 1 Build and Observe the Network

Phase 1 covers only concepts already taught in the course. You are not expected to implement subnetting, NAT, VLANs, or routing protocols unless your faculty explicitly adds them as an extension.

6.1 Course Topic Mapping

Every task in Phase 1 directly connects to a topic covered in your lectures. Use this table to understand what concept is being tested by each task.

Course Topic	What You Must Show in This Project
Moving Data Through the Core	Trace how a client request leaves your machine, crosses the LAN, reaches the edge service, and returns. Capture this in Wireshark.
OSI vs TCP/IP Model	Map DNS (Application), HTTP (Application), TLS (Session/Transport), TCP/UDP (Transport), IP (Network), and Ethernet (Link) to the correct layers.
Devices, Topologies, Cloud Concepts	Draw your local topology and relate each role (DNS server, load balancer, backend) to its cloud equivalent.
HTTP/1.1, HTTP/2, REST	Build a small REST API. Demonstrate HTTP/1.1; show HTTP/2 if your nginx supports it. HTTP/3 is explanation-only.
HTTPS and TLS	Terminate TLS at the nginx edge. Capture and explain the TLS handshake packets and certificate validation.
DNS and Route 53 Concepts	Create a private DNS name and configure at least two client machines to resolve through your team DNS server.

Transport Layer; Ports; TCP/UDP	Identify service ports, socket pairs, and capture the TCP three-way handshake before application data is exchanged.
Reliable Data Transfer; TCP Flow Control	Use Wireshark to identify sequence/acknowledgement numbers and explain TCP reliability at a conceptual level.
Email Protocols; CDNs and Caching	Email protocols are explanation-only. Caching is demonstrated using HTTP Cache-Control headers and conditional (304)
Cloud Load Balancing	Configure nginx as a local load balancer. Relate your setup to the role of cloud load balancers (AWS ALB, GCP Load Balancer).

6.2 Phase 1 Mandatory Build Tasks

All seven tasks below are mandatory. Every task has a clear success condition and a list of evidence you must collect and be ready to show during evaluation.

Task A Establish the Private LAN

Connect all team machines to the same private Wi-Fi or LAN. This is the foundation – nothing else works until this is verified.

- Connect all available team Macs to the same private Wi-Fi or LAN segment.
- Record each Mac's private IPv4 address, subnet mask/prefix, default gateway, active interface name, and MAC address.
- Verify basic reachability between every pair of machines using ping.
- Create a topology diagram clearly showing the four machine roles and how they are connected.

Task B Configure a Private DNS Server

Mac 1 runs the private DNS service. We recommend dnsmasq for Phase 1 because it is lightweight and easy to configure on macOS.

Create DNS records that resolve:

```
app.teamX.test    →    <Mac 2 private IP>
api.teamX.test    →    <Mac 2 private IP>
```

- Configure at least two other Macs to use Mac 1 as their DNS resolver (change System Preferences → Network → DNS).

- Verify resolution using `dig app.teamX.test` or `nslookup app.teamX.test` from a client machine.
- Access the final application by name, never by directly typing the IP address.
- Be prepared to explain the difference between DNS resolution (finding the IP) and the TCP/HTTPS connection that follows.

Task C Build Two Simple Backend Services

Mac 3 and Mac 4 each run a small HTTP REST backend. Keep the application code minimal the networking configuration is what matters here, not the app's features.

Each backend must respond to these endpoints:

Endpoint	Expected Response
GET /	A basic page or JSON object confirming the service is running.
GET /api/status	JSON with status and a clear backend identifier (e.g., { "backend": "A", "status": "ok" }).
Response Header	X-Backend: A (or X-Backend: B) so repeated requests show which backend served the response.

- Backends must listen on a LAN-accessible interface do NOT bind to 127.0.0.1 only or other machines cannot reach them.
- Use fixed, known TCP ports: Backend A on port 3001, Backend B on port 3002 (or faculty-agreed ports).

Task D Configure the Edge Reverse Proxy and Load Balancer

Mac 2 runs nginx and acts as the single public entry point. Clients never connect directly to the backends they always go through the edge.

```
Client → app.teamX.test → Mac 2 (nginx, port 443/8443)
      |
      |→ Mac 3 (Backend A, port 3001)
      |→ Mac 4 (Backend B, port 3002)
```

- Use round-robin load balancing (or another clearly explained strategy such as `least_conn`).
- Verify that repeated requests to the same domain name are served alternately by Backend A and Backend B.
- Be able to explain why the client never needs to know the backend IP addresses.

Task E Add HTTPS / TLS

The edge service must serve traffic over HTTPS using a valid certificate. The goal is to understand TLS termination, certificate trust, and the handshake not just to make the browser padlock appear.

- Create a self-signed certificate for app.teamX.test using OpenSSL, or use a local CA (e.g., mkcert) with faculty approval.
- Configure nginx to terminate TLS on port 443 (or 8443).
- Add your local CA certificate to the trust store of every client Mac so the browser or curl does not warn about an untrusted certificate.
- The final demonstration must not rely on bypassing certificate validation (no -k flag in curl for the demo).
- Be prepared to explain the TLS handshake steps: ClientHello → ServerHello → Certificate → Key Exchange → Finished.

Task F Demonstrate HTTP Caching Behavior

At least one endpoint must demonstrate proper HTTP cache headers so that repeated requests can be served from cache or validated efficiently.

- Return a Cache-Control header from at least one endpoint (e.g., Cache-Control: max-age=60).
- Optionally return an ETag header for conditional request support.
- Show the response headers using curl -I <url> or browser developer tools (Network tab).
- Demonstrate a repeated request showing that the client reuses cached content, or a conditional request returning 304 Not Modified.
- Explain the difference between a fresh cache hit, a conditional request, and a full new request.

Task G Capture the Complete Protocol Flow

This task ties everything together. Using Wireshark, tcpdump, curl, and browser tools, you must collect evidence that every layer of the stack is working correctly for a single client request.

Layer / Event	What You Must Show and Explain
DNS	A query from the client for app.teamX.test and the DNS response containing Mac 2's IP address.
TCP Handshake	The SYN → SYN-ACK → ACK sequence before any application data is sent. Record source and destination ports.
TLS Handshake	The ClientHello, ServerHello, Certificate, and ChangeCipherSpec packets. Show that application data is encrypted.
HTTP Headers	Request and response headers using curl -v or browser developer tools. Explain why the payload is encrypted in the Wireshark capture.
Load Balancing	Multiple repeated requests showing responses from both Backend A and Backend B (visible in the X-Backend header).
Port Identification	The client's ephemeral source port and the server's well-known destination port for each layer (53/UDP for DNS, 443/TCP for HTTPS).

6.3 Phase 1 Required Failure Demonstrations

You must deliberately break your setup in the following ways and explain what you observe and why it happens. This proves you understand the system, not just that you followed setup instructions.

Failure Scenario	Expected Observation and Explanation
Wrong DNS server configured on a client	Name lookup fails even though the machines still have direct IP connectivity. Shows DNS and IP layers are independent.
DNS record points to a wrong IP address	DNS resolution succeeds but the client is sent to the wrong destination. Shows DNS is a directory, not a connection.
One backend is stopped	The edge should continue serving requests through the remaining backend (if nginx health-check or failover is configured).
Both backends are stopped	DNS and TLS may still work at the edge, but the application returns a clear upstream failure (502 Bad Gateway). Shows where the edge ends and the backend begins.
Wrong destination port on the client	The host is reachable but the TCP connection to the service port fails. Shows ports and IP addresses are separate identifiers.

7. Phase 2 Harden, Recover, and Troubleshoot

Phase 2 extends the same network you built in Phase 1. You do not rebuild from scratch. The goal is to make your network more resilient, more secure, and more diagnosable. Every extension below is mandatory unless marked optional.

7.1 Mandatory Extensions

Extension A Backup DNS Resolver

- Run a second DNS resolver on another Mac (not Mac 1) with the same project DNS records.
- Configure client Macs to list both the primary (Mac 1) and backup DNS server addresses.
- Stop the primary DNS service (stop dnsmasq on Mac 1) and show that clients continue to resolve names using the backup.
- Explain the difference between a DNS service failure (name resolution stops) and an application server failure (resolution works, but the service is down).

Extension B DNS TTL and Controlled Record Change

- Set a short, clearly visible TTL (e.g., 30 seconds) on a project DNS record.
- Resolve the name from a client, then change the DNS record to point to a different IP.
- Observe the client still getting the old (cached) answer until the TTL expires, then getting the new one.
- Demonstrate flushing the DNS cache manually and observing the immediate change.
- Explain how this behavior relates to DNS-based service migration and traffic cutover in production systems.

Extension C Service Isolation (Backend Firewall Rules)

- Restrict direct access to backend ports (3001 and 3002) so that only Mac 2 (the edge) can reach the backends.
- Use macOS firewall / pf rules, or a faculty-provided rule template.
- Demonstrate that Mac 2 (nginx) can still reach the backends while a client Mac cannot directly connect to port 3001 or 3002.
- Maintain a rollback copy of any firewall rules and restore the original configuration after the demonstration.

Extension D High-Availability Failover Behavior

- Configure nginx to detect backend unavailability and route only to the healthy backend.
- Stop Backend A (Mac 3) and demonstrate that all requests continue to be served correctly through Backend B (Mac 4).
- Restart Backend A and demonstrate that load balancing resumes distributing across both backends.
- Identify clearly which component in your current design is still a single point of failure (the edge nginx on Mac 2) and explain what would be needed to eliminate it.

Extension E Controlled Edge Migration (DNS-Based Cutover)

- Configure a standby nginx on another Mac (can be Mac 3 or Mac 4 temporarily) with the same configuration.
- Update the DNS record for app.teamX.test to point to the standby edge machine.
- Observe TTL effects: show that existing cached clients briefly still hit the old edge while new lookups go to the standby.
- This can be done without any new physical hardware.

Extension F Faculty-Injected Troubleshooting Challenge

- During the evaluation, the faculty will introduce one configuration fault into your running system.
- Your team must identify the affected layer (DNS? TCP? TLS? Application?) and isolate or fix the issue.
- Use systematic diagnosis: check name resolution first, then TCP connectivity, then TLS, then the application layer.
- Explain your reasoning out loud as you diagnose partial credit is awarded for correct methodology even if the fix is incomplete.

8. Final Demonstration Sequence

The final demonstration follows a fixed sequence. Every team must prepare to show all 11 steps. The faculty may stop at any step to ask questions.

#	Demonstration Step	What the Evaluator Expects to See
1	Show topology and IP/service inventory	Display your network diagram, IP table, and service map. Every machine's role must be clear.
2	Confirm all machines are on the private LAN	Run ping between all machines. Show each machine's IP address.
3	Resolve the private domain from a client	From a client Mac, run <code>dig app.teamX.test</code> show it resolves to Mac 2's IP using your team DNS server.
4	Open the service over HTTPS using the domain name	Open <code>https://app.teamX.test</code> in a browser or curl. No certificate warnings. No IP address in the URL.
5	Show load balancing across both backends	Run repeated curl requests. Show X-Backend: A and X-Backend: B alternating in the response headers.
6	Show Wireshark evidence of DNS, TCP, and TLS	Open a saved Wireshark capture. Point to the DNS query/response, TCP three-way handshake, and TLS handshake packets.
7	Show HTTP headers and caching behavior	Use curl -I to show Cache-Control headers. Demonstrate a 304 Not Modified or cache hit.
8	Fail one backend and prove the service continues	Stop Mac 3 (Backend A). Show that requests still succeed through Backend B only.
9	Demonstrate Phase 2 DNS and resilience behavior	Show backup DNS failover, TTL/caching, or DNS-based cutover (Extension A/B/E).
10	Diagnose the faculty-injected fault	The faculty introduces one fault. Your team diagnoses and fixes or isolates it layer by layer.
11	Individual viva questions	Every team member answers individual questions. Answers must be from personal understanding, not from reading notes.

9. Student Deliverables

All deliverables must be organised and ready before the evaluation slot. The evaluator should be able to find any piece of evidence within 30 seconds.

Deliverable	Due At	Required Contents
Architecture Document	Phase 1 (before Phase 1 review) + updated for Phase 2	Network topology diagram · machine roles and IP/service table · request-flow diagram showing each protocol layer.
Configuration Bundle	Phase 1 + Phase 2 additions	dnsmasq config · nginx config (Phase 1 and Phase 2) · TLS certificate setup notes · backend launch instructions · firewall rules if used.
Backend Source Code	Phase 1	Complete code for both backend applications and any helper scripts. Hosted on GitHub or shared as a zip.
Evidence Folder	Phase 1 + Phase 2 additions	Screenshots or exports of: DNS resolution (dig/nslookup) · curl/browser output with headers · Wireshark captures for DNS, TCP handshake, TLS handshake · HTTP caching demo · all failure scenario
Phase 1 Checkpoint	Phase 1 review date	Working live system + brief structured demonstration + each team member prepared to explain any component.
Phase 2 Final Report	Phase 2 review date	What changed from Phase 1 · resilience tests and results · troubleshooting findings · learning summary (one paragraph per extension).
Final Presentation	Final evaluation date	Live demonstration following Section 8 sequence · every team member must participate and answer individual viva questions.

10. Marks Breakdown

The project is evaluated in two phases totalling 100 marks. Review 1 covers Phase 1 build tasks. Review 2 covers Phase 2 extensions, resilience, and the final integrated demonstration.

Review	Area	Marks	Notes
Review 1	LAN Setup + Private DNS Configuration (Task A + B)	10	Topology, ping, DNS records, client resolver setup.
	HTTP/REST Backends + Reverse Proxy + Load Balancing (Task C + D)	10	Both backends running, nginx upstream, X-Backend header visible.
	HTTPS / TLS Correctness and Explanation (Task E)	8	Certificate setup, nginx TLS config, TLS handshake captured and explained.
	Packet Analysis and Protocol Flow Evidence (Task G)	7	Wireshark captures: DNS, TCP handshake, TLS, ports identified.
	HTTP Caching and Transport Layer Understanding (Task F)	5	Cache-Control shown, 304 or cache hit demonstrated.
	Individual Viva Phase 1 Concepts	10	Per student: ability to explain any component of the Phase 1 build.
Review 1 Total		50	40 marks team + 10 marks individual viva
Review 2	Phase 2 Resilience and HA Failover (Extensions A, B, D)	15	Backup DNS, TTL behavior, HA failover demonstrated.
	Service Isolation and Edge Migration (Extensions C, E)	10	Backend firewall rules, DNS cutover, TTL effects shown.
	Troubleshooting Challenge (Extension F)	10	Correct layer identified, systematic diagnosis, partial credit for methodology.
	Final Report Quality and Documentation	5	Phase 2 report: changes, test results, learning summary.
	Individual Viva Phase 2 Concepts + Full System Understanding	10	Per student: ability to explain the complete system end to end.
Review 2 Total		50	40 marks team + 10 marks individual viva
TOTAL	Both Phases Combined	100	